

Welcome!

We'll get started shortly. Please take the Zoom poll in the meanwhile!

CS 49 Week 7

Surajit A. Bose

How to get hold of me / get help from other resources

- Surajit's office hours
 - Fridays 12 noon–1p, directly after section
 - By appointment on [Zoom](#)
 - Post on the [section forum](#), usually get a response within 2 hours
 - Email bosesurajit@fhda.edu or Canvas inbox, 24 hr turnaround
 - The [github repo](#) has section materials, starter code, and solutions
-

- Canvas inbox for Lane
- The [main forum](#) for the course on the Code in Place site
- [Lane's office hours](#)
- [Online](#) or [in-person](#) tutoring (Room 3600)
- [One-on-one code reviews](#) with Sarah or Lia

Agenda

- Goal: to understand functions in Python, including passing data into and getting data out of functions
- Function input: parameters, arguments
- Function output: **return** statements and values
- Functions as black boxes
- Worked example: [In range](#)
- Revised [coding template](#)
- Section problem: [Medical Test Simulator](#)

Review: Decomposition and Problem Solving



The process of problem solving: stepwise refinement

- First, **understand** the pre- and postconditions
- While **designing the solution** , identify the repeatable, self-contained building blocks that will be the functions: **decompose** thoroughly
- When coding, assume the functions are done
 - Write out the function headers and the pre- and postconditions
 - Use the **pass** keyword for the bodies
- Assemble the functions in **main()** to solve the overall problem
- **Implement** each building block by writing out its function body
- **Test** iteratively and **refine** the solution:
 - Check that each function works as expected
 - Check that all functions put together in **main()** solve the entire problem as expected

Functions in Python

In range

- In **main()**, prompt the user for three numbers, **n**, **low**, and **high**
- Call **in_range()** to check whether **n** is in the range between **low** and **high**, inclusive
- Back in **main()**, print whether **n** is in range or not
- User input is in **blue**:

```
n: 5
low: 2
high: 8
n is in range!
```

```
n: 5
low: 1
high: 3
n is not in range...
```

Function input

- **main()** needs to get some values as user input, then pass them to **in_range()**
- Question: How does **main()** pass those numbers on to **in_range()**?
- To specify what input data it expects, a function has zero or more **parameters**
- Example of function with input parameters: [Print multiple](#)
- The function header **def print_multiple(message, repeats)** specifies that **print_multiple()** expects two values, **message** and **repeats**, as input
- The **print_multiple(message, repeats)** instruction in **main()** sends the expected values as **arguments** to the function
- The function uses the received data to do its work, i.e., to print out the message the appropriate number of times
- So, **main()** will need to send three values as arguments to **in_range()**

Function output

- Every time **main()** has numbers to check, it calls **in_range()**
- **n**, **low**, and **high** are the parameters or expected input to **in_range()**
- **in_range()** then tells **main()** whether or not the number is in the given range
- Question: How does **in_range()** pass the result on to **main()**?
- To get data out: a function **returns** zero or one values as output
- Example of function with return value: [Is odd](#)
- The **return** value or the output from **is_odd()** is a boolean: **True** if the value is odd, **False** otherwise
- Aside: notice that the variable whose value is passed as an argument to **is_odd()** does not have the same name as the parameter in the header

Function calls and control flow

- A function is **called** from within another function, the **calling function** or **caller**
- When calling the called function, the caller passes in appropriate **arguments** for the expected parameters
- The called function does its work, and if there is a return value, sends the value to the caller
- Execution returns to the caller, which handles the return value if there is one
- Here, **main()** is the caller and **in_range()** the called function
- The values of **n**, **low**, and **high** are arguments passed from **main()** to **in_range()**
- The value returned from **in_range()** is used to determine what gets printed out
- We now have enough information to complete the [In range](#) section problem!

Function structure

- The function header has the keyword **def**, the function name, and parentheses with the parameter list, followed by a colon :

```
def in_range(n, low, high):
```

- Indented below the header is the function body, which does the required work using the input values:

```
    return low <= n <= high
```

- The function has zero or more return statements depending on its work.
- Examples of a function with multiple return statements is on the next slide
- Note that only one return statement will ever execute

Functions as black boxes

- The function is a black box; the caller does not know how the called function does its work.
- E.g., `in_range()` could be defined in these alternative ways (among others):

```
def in_range(n, low, high):  
    if n < low or n > high:  
        return False  
    return True
```

```
def in_range(n, low, high):  
    if n >= low and n <= high:  
        return True  
    return False
```

- From the standpoint of the calling function, it does not matter what happens inside the called function as long as the result is as expected

Advantages of using functions

- The three Rs: readability, repeatability, reliability
- Compare these two programs that do exactly the same thing
 - [logical_operators.py](#)
 - [logical_operators_func.py](#)
- The program that uses a separate function to evaluate and print the passed in arguments is much shorter, clearer, and cleaner than the program that does everything in **main()**
- There is also less risk of error in the program that uses a separate function
- A template for writing code that uses functions with parameters and return values is at the [github repo](#)

Section Problem: Medical Test Simulator

Medical Test Simulator (Slide 1 of 4)

- Given:
 - A population of 10,000 individuals
 - An infection rate of 1% for a disease
 - A diagnostic test with 99% accuracy
- ... what percentage of positive results from the test will be false positives?

```
Number of people: 10000
Test accuracy: 0.99
Infection rate: 0.01
True positives: 93
False positives: 113
False negatives: 1
True negatives: 9793
54.85436893203883% of positive tests were incorrect
```

Notes:

- User input is in blue
- With probabilistic simulations, results will be different each time the test is run

Medical Test Simulator (Slide 2 of 4)

- Starter code is at the [github repo](#)
- In **main()**, prompt the user for these three numbers:
 - number of people
 - test accuracy
 - infection rate
- In **simulate_tests()**:
 - Use the given probabilities and simulate a diagnostic test on each individual
 - Randomly determine whether the individual is infected
 - Randomly determine whether the test is accurate

Digression: Simulating a probabilistic outcome

- The assignment provides this hint: the expression `random.random() < prob` evaluates to `True` with probability `prob`
- Explanation:
 - The range 0 through 9 comprises 10 integers
 - Of those, 8 integers (0 through 7 inclusive) are below 8
 - So if we run `random.randint(0, 9)`, there is an 8 in 10 chance that the integer we get will be below 8. Eight in 10 is 8/10 or 80%.
 - Likewise, `random.random()` returns a float from 0 up to but not including 1
 - Therefore, given a probability of 0.8, 80% of the time the random float will be below 0.8

Medical Test Simulator (Slide 3 of 4)

- In **simulate_tests()**, contd.:
 - Calculate and print these four values:
 - true positives
 - false positives
 - false negatives
 - true negatives
 - Return the proportion of incorrect positive results
- Back in **main()**:
 - Print out the percentage of incorrect positive tests

Medical Test Simulator (Slide 4 of 4)

- What type(s) should user input be cast to in **main()**?
- How many parameters should **simulate_tests()** expect? Of what type(s)?
- How should we calculate the four necessary values of true and false positives and negatives?
- Do we need a loop? If so, for what?
- What is the formula for the **return** value, i.e., the proportion of incorrect positive results?
- How can we display this as a percentage in **main()**?

That's all, folks!

Next up: Lists!

Bonus Slides

[Medical Test Simulator](#) and Data Science

Confusion matrices

- In data science, this sort of tally is called a **confusion matrix**

	Diagnosed as infected	Diagnosed as not infected
Actually infected	93 (true positives)	1 (false negative)
Actually not infected	113 (false positives)	9793 (true negatives)

Number of people: **10000**

Test accuracy: **0.99**

Infection rate: **0.01**

True positives: **93**

False positives: **113**

False negatives: **1**

True negatives: **9793**

54.85436893203883% of positive tests were incorrect

This test has good **recall** or **sensitivity**:

93 out of 94 infected individuals were identified (98.94%).

This test has poor **precision**: 93 out of 206

individuals identified as infected were infected (45.15%)

This test has good **specificity**: 9793 out of

the 9906 uninfected individuals were correctly identified (98.86%)

The base rate fallacy

- The assignment asks: "As you can see, the results are pretty surprising: nearly 55% of people told they had the disease didn't actually have it. Intuitively, why might this be? Is our result just anomalous?"
- This is the accuracy paradox, aka the false positive paradox or [base rate fallacy](#)
- Given the infection rate of 1% in a population of 10,000:
 - 100 people are infected
 - 9,900 are uninfected
- Given the test accuracy rate of 99%:
 - Of the 100 infected people, we will likely have 99 true positives and 1 false negative
 - Of the 9,900 uninfected people, we will likely have 9,801 true negatives and 99 false positives
 - Of the total 198 positive results, likely half or 50% of them will therefore be false positives!
- Reminder: with probabilistic simulations, results will be different each time and the results won't be exactly as mathematically predicted